Presentation 3

- Links to presentation(s) and code(s) on GitHub

Presentation Recording Dashboard Code

- What did you do?

I created an interactive dashboard to help investigate how calls end up getting redirected to the 8300 line at the end. More specifically, for all the calls that ended with the 8300 number as the Called number, I looked at the Redirecting number in the last leg and the correspond Redirect reason.

- 1 stacked bar plot:
  - Distribution of Redirecting Numbers at the Ends of Calls Ending With 8300
    - x = Redirecting number
    - y = Count of calls (or Percent of Calls that end with 8300)
    - color = Redirect reason
- 1 slicer:
  - Range of columns to show
    - interactive slider to decide which columns to show in the plot in terms of their ranking by count
    - e.g., a range from 1 to 7 will show the top 7 most commonly occurring redirecting numbers
- phone number descriptions for reference (only has the top 10 most common redirecting numbers)

In Python, I extracted the sequence of Called numbers, Redirecting numbers, and Redirect reasons for every single call session. My method of extraction successfully ran through the entire AllCalls dataset in a reasonable amount of time. From each sequence, I used the last leg to get counts for how often any redirecting number redirects to 8300. Additionally, for each redirecting number, I also retrieved the distribution of redirect reason. Altogether, I had a table that essentially has 2 hierarchical categories and a count column.

The table also contains a column representing the proportion of all calls ending in 8300 that correspond to each redirecting number & redirect reason combination. From that, a column with percentage strings was made to display percents in the dashboard.

My analysis this time was largely individual.

Miscellaneous insight:

- While exploring the data (the following is not part of my final uploaded Jupyter notebook), I found out that there are a surprising number of calls that have the exact same sequence of Called numbers and Redirecting numbers (i.e., many calls appear to have the same journeys). The following example occurs 22379 times among all calls that end in 8300:

Called number	Redirecting number
13123411070	
13124235938	13123411070
13124235938	13123411070
13123478300	13124235938
13123478300	13124235938

- How does it help the project?

There is no actionable recommendation at this time, but there are some insights from my analysis that could be useful moving forward. So far, my dashboard essentially only confirms the idea that 8300 is often the voicemail line that other lines redirect to when they are not available. For voicemail lines like 6344 (Bankruptcy Helpdesk VM), the redirect reason is always Unconditional (i.e., those lines automatically redirect to 8300), showing how 8300 is connected to voicemail. However, there is not enough information to identify the rates at which redirecting numbers are redirecting to 8300 at the ends of calls, which would help identify which lines are ineffective. The identification of which lines appear to be struggling more than other lines would be more informative for potential recommendations.

- Issues faced

I faced several issues in my dashboard making process that ultimately led me to forgo some of the dashboard features I originally wanted.

- One issue was getting the legend to appear properly when using a toggle for whether or not to show redirect reasons (i.e., have the bar plot be stacked). Normally, an unstacked bar plot would be made by leaving the legend field blank. However, you cannot dynamically choose between leaving that field empty versus using Redirect reason in that field. I ultimately decided to leave the plot as a stacked bar plot with bar totals displayed above the bars.
- Another issue was creating a toggle that switches between displaying the values in the plot as raw counts versus as percentages. There were no straightforward ways to get proportions to be displayed in a percentage format without representing percents as strings. However, representing percents as strings complicates the sorting process of displaying the biggest bars at the left of the plot. I ultimately just created a column of percents formatted as strings in Python, and I used that column as "Detail" labels on the plot.

- Next steps

To identify which lines are leading to unsuccessful calls more than others (and thus might be struggling to handle the call load):

- I will look at the overall distribution of redirect reasons for each redirecting number. This should be fairly straightforward as this is very similar to what I did for my previous dashboard, substituting Called number with Redirecting number instead. This can give a better idea of how often lines are redirected from not being answered or being unavailable.
- For each redirecting number, I could look at every call session that includes that number, and identify common trends about the context that they redirect in. This can be done by, for some instance of a redirecting number, comparing what lines precede it and what lines come after.
- Including some more columns beyond just Called number, Redirecting number, and Redirect reason could be informative. Specifically, User type and Client type could provide more information about what is happening in a call.
- I can also explore other potential numbers for calls to end on, such as the 1070 line. This will be done by just replacing the 8300 line with whatever other number in my code as my code is generalizable.

Some of these are essentially continuations of the next steps I came up with last presentation.

- References

For the code, it is assumed that you have the combined csv file with the AllCallsData, which is created using Krish's AllCallsData_data_import_code.

Presentation 2

- Links to presentation(s) and code(s) on GitHub

Presentation Dashboard Code

- What did you do?

Individual:

- I completed the objective of creating a dynamic visualization of the distributions of each of the 3 reason columns, allowing for filtering by Called number, date range, and day of the week.
    - 3 bar plots:
        - Distribution of Original reason (x = Reason type, y = Count of Correlation ID)
        - Distribution of Redirect reason (x = Reason type, y = Count of Correlation ID)
        - Distribution of Related reason (x = Reason type, y = Count of Correlation ID)
    - 3 slicers for filtering:
        - Date range
            - interactive slider to count only rows that happen between 2 dates (MM/DD/YYYY)
        - Filter by called number
            - interactive multi-select checkboxes to count only rows with the selected Called number
            - 20 checkboxes (not counting Select all):
                - 19 called numbers
                    - the 17 numbers with descriptions in the Clarifications file
                    - the 2 numbers without descriptions but are part of the top 10 most frequently occurring numbers
                - (Blank)
                    - represents the data that correspond to all the Called numbers besides the 19 listed
        - Filter by day of the week
            - interactive multi-select checkboxes to count only rows occuring on any given days of the week (adjusted to Chicago time)
            - 7 checkboxes (not counting Select all); 1 for each day of the week
- In Python:
    - Reformatted the combined AllCallsData to be more easily used by Power BI (fixing data types, removing irrelevant columns to reduce file size)
    - Used the Clarifications file with the phone number descriptions to create a table that links phone numbers to descriptions

Collaborative:

- Tatum and I were the two people who focused on the objective relating to original/related reason for redirection.
- The main way we collaborated was through sharing insights and ideas about the AllCallsData (e.g. how the reason columns can be used together to understand how a call progresses). However, we worked on our dashboards individually.

- How does it help the project?

There is no feasibly actionable recommendation from my analysis at this time. One idea I had is to reallocate agent availability towards phone numbers that often result in “NoAnswer”, which would help reduce the number of calls that end in voicemails. The main issue with this recommendation is that there may be cost or logistical constraints that make reallocation difficult in practice.

With the filtering, only the Called number filtering yielded interesting differences. I did not find anything particularly interesting when filtering by date range and days of the week, holding Called number constant.

- Issues faced

There were no significant issues in my analysis process this week.

- Next steps

So far, the distributions by themselves are mainly effective at indicating how certain Called numbers tend to be reached in calls. For example, phone numbers that are mainly for transferring and moving callers around in the phone system (e.g. 13125068646 = Transfers to the English main menu) would have original reason distributions dominated by “Deflection”.

The most frequent number (13123478300), which represents internal voicemail, has a redirect reason distribution dominated by “NoAnswer”. This indicates that 8300 is typically reached when the previous number did not answer, which makes sense for the 8300 as the final voicemail number at the ends of failed calls. This would mean that a call having its last leg be at 8300 could be used as an indicator of call failure.

A next step would be to identify and analyze common trends in how the reason columns, User type, and Client type behave right before the last leg of calls that end with 8300. These columns altogether provide crucial information about what is happening in each leg and why one leg leads to another, so analyzing them specifically would be helpful. These trends can be used to identify more specifically where agent availability is more likely to be the main issue rather than menu inefficiencies.

- References

For the code, it is assumed that you have the combined csv file with the AllCallsData, which is created using Krish’s AllCallsData_data_import_code.

Presentation 1

- Links to presentation(s) and code(s) on GitHub

Google Slides Dashboard Code

- What did you do?

New metric proposal: Self-Service Success Rate
Main visualization: Count of Ending User Type For Customer Calls

1. **Preliminary visualizations with Power BI using the AllCallsData**

I collaborated with my team to create basic Power BI visualizations. This involved some simple data cleaning within Power BI (e.g. converting Called number column to Text data type to resolve import errors). Although not every visualization is of equal importance, I created 5 visualizations total:

- Count of Redirect Reason (bar chart)
- Average Duration by Redirect Reason (bar chart)
- Average Duration by User Type (bar chart)
- Proportion of Answer Indicators By User Type (stacked bar chart)
- Count of Ending User Type For Customer Calls (bar chart)

The first 4 visualizations are exploratory, whereas the last visualization (Count of Ending User Type For Customer Calls) is a step towards visualizing my proposed metric.

2. **EDA and data processing for AllCallsData**

Individually, I spent time investigating how customer calls can be identified using columns in the data. I researched what the Inbound/Outbound trunk, Direction, Call type, and Client type columns represented, and I came to the conclusion that incoming calls from customers would satisfy the following conditions in the earliest leg of the call:

- Inbound trunk = NA
- Outbound trunk = wcc_[something]
- Direction = TERMINATING
- Call type = SIP_INBOUND
- Client type = WXCC

This was used in my data cleaning code in Python. I removed columns that were exclusively NaN, converted all phone number columns to string instead of integer/float, and converted columns with mixed data types to entirely strings. Then, I grouped the data by Correlation ID. Using the conditions I came up with to detect calls that are specifically inbound customer calls, I filtered the Correlation IDs. This yielded a collection of data frames that each represent one customer call.

I also explored the User type column, which can indicate if a call reached a live agent (User type = User), if a leg corresponds to a customer still navigating the submenu (User type = AutomatedAttendantVideo), if a call ended with a voicemail (end leg's User type = VoiceMailRetrieval), etc.

For each possible User type value, I created a small table that contains the counts of how many customer calls have that User type value in the ending leg. This table was fed into Power BI to create the main visual.

3. New metric: Self-Service Success Rate

This metric is calculated by how many calls that never involved a live agent did NOT end with a customer abandoning the call. This metric would be important in showing how effectively customers will be able to complete their task without needing a live agent. This can help identify ways to reduce the call load on live agents, streamlining the call process.

In the data, this would involve steps like looking at what calls never have a leg with User type = User. Identifying whether a call ended with a customer abandoning the call could involve using the Original/Redirect reason, Call outcome, Call outcome reason, and Answer Indicator columns.

Mathematically: # of self-service calls that did not end in abandonment / total # of self-service calls.

- How does it help the project?

The preliminary visualizations were to help us get the hang of Power BI as a tool and to understand the data better with hands-on work.

For my main visualization, it helps the project as it depicts the distribution of call outcomes for customer calls specifically. Although there are more columns that detail the exact way in which calls end, the User type column is especially useful for this as it shows info like whether a call ended in voicemail or a live agent. With this kind of info, we can identify points of improvement in the phone system if, say, calls ending in voicemail is the dominant way in which calls may be considered as unsuccessful.

Looking at the visualization, it is visible that the User column (i.e. calls that ended in live agents) is noticeably smaller than the columns for the AutomatedAttendantVideo (i.e. calls that ended with the customer still in the submenu) and for the VoiceMailRetrieval (i.e. calls that ended with voicemail). This suggests that there is a relatively high rate of customers leaving while trying to navigate the submenu, and calls that end with a live agent are rare. This can serve as a starting point for identifying where issues are present in call journeys.

For my metric, it helps the project by allowing us to identify weak points in the submenu/routing system, and the ultimate goal would be to maximize the metric as that would mean less call loads for live agents. My metric would allow us to also gauge how well the company is performing in that regard. If the metric is low (i.e. high rate of abandonment without live agents), then identifying at what point those calls are abandoned could be helpful in determining a more specific point of improvement.

My data cleaning process will also be very helpful going forward as it organizes the data into individual calls that can be analyzed and compared. This will help us in getting a better understanding of how different call journeys are represented differently by the data. It will also make it easier to get info that cannot be easily obtained by simply looking at each row/leg separately.

- Issues faced (if any)

Errors in importing the data to Power BI. Inability to share dashboards further complicated the issue my group had with me being the only one with a Windows PC. Trying the run the grouping code for the full AllCallsData ended up taking over 30 minutes,

- Attempts to resolve issues (if any)

Power BI import errors were resolved by simply converting a column (“CalledNumber”) to strings. The Power BI computer issue with my groupmates all having Macs was resolved by trying to use library computers. We

also ended up just making separate dashboards for our individual metrics and put aside the idea of sharing one pbix file for now. For the excessive runtime of my code, I only used the August 2025 data for now.

- Issues resolved (if any)

Power BI import errors were fixed. We managed to make something work with Power BI despite our different computer systems (and we hope to have Power BI Pro access soon). The excessive runtime for my code on the entire AllCalls dataset is still unresolved.

- Next steps

The biggest next step I have is to identify ways to use the AllCalls data to determine whether a call was abandoned or not and whether a call can be considered a success. I have some specific columns in particular to research and understand: Original/Redirect reason, Call outcome, Call outcome reason, and Answer Indicator. These appear to be potentially useful as they all give information about the state of a call at any given leg, and a call that was abruptly ended by the customer could be indicated by these columns. After understanding these columns better, I plan to use them in a conditional statement that can essentially label call sessions as successful or abandoned using Python code.

Another major next step is figuring out ways to rewrite the grouping code that will make splitting the combined AllCalls dataset into separate customer calls take less time to run. I suspect that using a for loop to loop over different dataframes with dozens of columns might be inefficient, so I think somehow reducing the size of the dataframes for the sake of just getting the correlation IDs that correspond to customer calls could help.

- References (Mention if you built up on someone else's work)

n/a